

TWO TRUTHS & A LIE

Creating and Using Interfaces

1. Java's capability to inherit from more than one class is called multiple inheritance.
2. All of the methods in an interface are implicitly `public` and `abstract`, and all of its data items (if any) are implicitly `public`, `static`, and `final`.
3. When a class inherits from another, the child class can use the nonprivate, nonoverridden members of its parent's class, but when a class uses an interface, it must implement its own version of each method.

The false statement is #1. The ability to inherit from more than one class is called multiple inheritance, but Java does not have that ability.



You Do It

Using an Interface

In this section, you create an `Insured` interface for use with classes that represent objects that can be insured. For example, you might use this interface with classes such as `Jewelry` or `House`. Also in this section, you extend `Vehicle` to create an `InsuredCar` class that implements the `Insured` interface, and then you write a short program that instantiates an `InsuredCar` object.

1. Open a new file, and type the following `Insured` interface. A concrete class that implements `Insured` will be required to contain `setCoverage()` and `getCoverage()` methods.

```
public interface Insured
{
    public void setCoverage();
    public int getCoverage();
}
```

2. Save the file as **Insured.java** and compile it.
3. Open a new file, and start the `InsuredCar` class that extends `Vehicle` and implements `Insured`:

```
import javax.swing.*;
public class InsuredCar extends Vehicle implements Insured
{
```

(continues)

(continued)

4. Add a variable to hold the amount covered by the insurance:

```
private int coverage;
```

5. Add a constructor that calls the `Vehicle` superclass constructor, passing arguments for the `InsuredCar`'s power source and number of wheels.

```
public InsuredCar()
{
    super("gas", 4);
    setCoverage();
}
```

6. Implement the `setPrice()` method required by the `Vehicle` class. The method accepts the car's price from the user and enforces a maximum value of \$60,000.

```
public void setPrice()
{
    String entry;
    final int MAX = 60000;
    entry = JOptionPane.showInputDialog
        (null, "Enter car price ");
    price = Integer.parseInt(entry);
    if(price > MAX)
        price = MAX;
}
```

7. Implement the `setCoverage()` and `getCoverage()` methods required by the `Insured` class. The `setCoverage()` method sets the coverage value for an insured car to 90 percent of the car's price:

```
public void setCoverage()
{
    coverage = (int)(price * 0.9);
}
public int getCoverage()
{
    return coverage;
}
```

8. Create a `toString()` method, followed by a closing brace for the class:

```
public String toString()
{
    return("The insured car is powered by " + getPowerSource() +
        "; it has " + getWheels() + " wheels, costs $" +
        getPrice() + " and is insured for $" + getCoverage());
}
```

(continues)

(continued)

9. Save the file as **InsuredCar.java** and compile it.
10. Create a demonstration program that instantiates an **InsuredCar** object and displays its values as follows:

```
import javax.swing.*;
public class InsuredCarDemo
{
    public static void main(String[] args)
    {
        InsuredCar myCar = new InsuredCar();
        JOptionPane.showMessageDialog(null,
            myCar.toString());
    }
}
```

11. Save the file as **InsuredCarDemo.java**. Compile and execute it. You will be prompted to enter the car's price. Figure 11-34 shows the output during a typical execution.

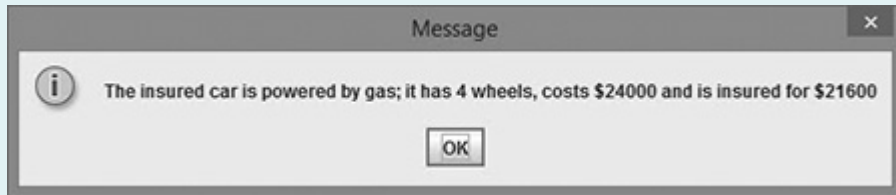


Figure 11-34 Typical output of the **InsuredCarDemo** program

Creating and Using Packages

Throughout most of this book, you have imported packages into your programs. As you learned in Chapter 4, a package is a named collection of classes; for example, the `java.lang` package contains fundamental classes and is automatically imported into every program you write. You also have created classes into which you explicitly imported optional packages such as `java.util` and `javax.swing`. When you create classes, you can place them in packages so that you or other programmers can easily import your related classes into new programs. Placing classes in packages for other programmers increases the classes' reusability. When you create a number of classes that inherit from each other, as well as multiple interfaces that you want to implement with these classes, you often will find it convenient to place these related classes in a package.



Creating packages encourages others to reuse software because it makes it convenient to import many related classes at once. In Chapter 3, you learned that if you do not use one of the three access specifiers **public**, **private**, or **protected** for a class, then it has default access, which means that the unmodified class is accessible to any other class in the same package.

When you create professional classes for others to use, you most often do not want to provide the users with your source code in the files that have .java extensions. You expend significant effort developing workable code for your programs, and you do not want other programmers to be able to copy your programs, make minor changes, and market the new product themselves. Rather, you want to provide users with the compiled files that have .class extensions. These are the files the user needs to run the program you have developed. Likewise, when other programmers use the classes you have developed, they need only the completed compiled code to import into their programs. The .class files are the files you place in a package so other programmers can import them.



In the Java programming language, a package or class library is often delivered to users as a **Java ARchive (JAR) file**. JAR files compress the data they store, which reduces the size of archived class files. The JAR format is based on the popular Zip file format.

If you do not specify a package for a class, it is placed in an unnamed **default package**. A class that will be placed in a nondefault package for others to use must be **public**. If a class is not **public**, it can be used only by other classes within the same package. To place a class in a package, you include a **package** declaration at the beginning of the source code file that indicates the folder into which the compiled code will be placed. When a file contains a **package** declaration, it must be the first statement in the file (excluding comments). If there are import declarations, they follow the **package** declaration. Within the file, the **package** statement must appear outside the class definition. The **package** statement, **import** statements, and comments are the only statements that appear outside class definitions in Java program files.

For example, the following statement indicates that the compiled file should be placed in a folder named `com.course.animals`:

```
package com.course.animals;
```

That is, the compiled file should be stored in the `animals` subfolder inside the `course` subfolder inside the `com` subfolder (or `com\course\animals`). The pathname can contain as many levels as you want.

When you compile a file that you want to place in a package, you can copy or move the compiled .class file to the appropriate folder. Alternatively, you can use a compiler option with the `javac` command. The `-d` (for *directory*) option indicates that you want to place the generated .class file in a folder. For example, the following command indicates that the compiled `Animal.java` file should be placed in the directory indicated by the `import` statement within the `Animal.java` file:

```
javac -d . Animal.java
```

The dot (period) in the compiler command indicates that the path shown in the package statement in the file should be created within the current directory.

If the `Animal` class file contains the statement `package com.course.animals;`, the `Animal` class file is placed in `C:\com\course\animals`. If any of these subfolders do not exist, Java creates them. Similarly, if you package the compiled files for `Dog.java`, `Cow.java`, and so on, future programs need only use the following statements to be able to use all the related classes:

```
import com.course.animals.Dog;
import com.course.animals.Cow;
```

Because Java is used extensively on the Internet, it is important to give every package a unique name. The creators of Java have defined a package-naming convention that uses your Internet domain name in reverse order. For example, if your domain name is `course.com`, you begin all of your package names with `com.course`. Subsequently, you organize your packages into reasonable subfolders.

Creating packages using Java's naming convention helps avoid naming conflicts—different programmers might create classes with the same name, but they are contained in different packages. Class naming conflicts are sometimes called **collisions**. Because of packages, you can create a class without worrying that its name already exists in Java or in packages distributed by another organization. For example, if your domain name is `course.com`, then you might want to create a class named `Scanner` and place it in a package named `com.course.input`. The fully qualified name of your `Scanner` class is `com.course.input.Scanner`, and the fully qualified name of the built-in `Scanner` class is `java.util.Scanner`.

TWO TRUTHS & A LIE

Creating and Using Packages

1. Typically, you place `.class` files in a package so other programmers can import them into their programs.
2. A class that will be placed in a package for others to use must be protected so that others cannot read your source code.
3. Java's creators have defined a package-naming convention in which you use your Internet domain name in reverse order.

The false statement is #2. A class that will be placed in a package for others to use must be `public`. If a class is not `public`, it can be used only by other classes within the same package. To prevent others from viewing your source code, you place compiled `.class` files in distributed packages.



You Do It

Creating a Package

Next, you place the `Vehicle` family of classes into a package. Assume you work for an organization that sponsors a Web site at `vehicleswesell.com`, so you name the package `com.vehicleswesell`. First, you must create a folder named `VehiclePackage` in which to store your project. You can use any technique that is familiar to you. For example, in Windows, you can double-click Computer, navigate to the device or folder where you want to store the package, right-click, click New, click Folder, replace “New Folder” with the new folder name (`VehiclePackage`), and press the Enter key. Alternatively, from the command prompt, you can navigate to the drive and folder where you want the new folder to reside by using the following commands:

- If the command prompt does not indicate the storage device you want, type the name of the drive and a colon to change the command prompt to a different device. For example, to change the command prompt to the F drive on your system, type `F:`.
- If the directory is not the one you want, type `cd\` to navigate to the root directory. The `cd` command stands for “change directory,” and the backslash indicates the root directory. Then type `cd` followed by the name of the subdirectory you want. You can repeat this command as many times as necessary to get to the correct subdirectory if it resides many levels down the directory hierarchy.

Next, you can place three classes into a package.

1. Open the **Vehicle.java** file. As the first line in the file, insert the following statement:
2. Save the file as **Vehicle.java** in the **VehiclePackage** folder.
3. At the command line, at the prompt for the `VehiclePackage` folder, compile the file using the following command:

```
javac -d . Vehicle.java
```

Be certain that you type a space between each element in the command, including surrounding the dot. Java creates a folder named `com\vehicleswesell\vehicle` within the directory from which you compiled the program, and the compiled `Vehicle.class` file is placed in this folder.

(continues)

(continued)



If you see a list of compile options when you try to compile the file, you did not type the spaces within the command correctly. Repeat Step 3 to compile again.

The development tool GRASP generates software visualizations to make programs easier to understand. A copy of this tool is included with your downloadable student files. If you are using jGRASP to compile your Java programs, you also can use it to set compiler options. To set a compiler option to `-d`, do the following:

- Open a jGRASP project workspace. Click the **Settings** menu, point to **Compiler Settings**, and then click **Workspace**. The Settings for workspace dialog box appears.
 - Under the FLAGS or ARGS section of the dialog box, click the dot inside the square next to the Compile option and enter the compiler option (`-d`). Then click the **Apply** button.
 - Click the **OK** button to close the dialog box, and then compile your program as usual.
4. Examine the folders on your storage device, using any operating system program with which you are familiar. For example, if you are compiling at the DOS command line, type **dir** at the command-line prompt to view the folders stored in the current directory. You can see that Java created a folder named `com`. (If you have too many files and folders stored, it might be difficult to locate the `com` folder. If so, type **dir com*. *** to see all files and folders in the current folder that begin with “com”.) Figure 11-35 shows the command to compile the `Vehicle` class and the results of the `dir` command, including the `com` folder.

```

C:\Java\VehiclePackage>javac -d . Vehicle.java
C:\Java\VehiclePackage>dir
Volume in drive C is Windows8_OS
Volume Serial Number is 92DA-E9D9

Directory of C:\Java\VehiclePackage

07/29/2014  04:08 PM    <DIR>          .
07/29/2014  04:08 PM    <DIR>          ..
07/29/2014  04:08 PM    <DIR>          com
07/29/2014  04:07 PM             756 Vehicle.class
07/29/2014  04:08 PM             688 Vehicle.java
                2 File(s)          1,444 bytes
                3 Dir(s)      883,646,189,568 bytes free

C:\Java\VehiclePackage>_
  
```

Figure 11-35 Compiling the `Vehicle.java` file in a package and viewing the results

(continues)

(continued)

Alternatively, to view the created folders in a Windows 8.1 operating system, you can swipe from the right edge of the screen, tap **Search**, start to type **File Explorer**, and tap **File Explorer**. Then search for the folder. Within the `com` folder is a `vehicleswesell` folder, and within `vehicleswesell` is a `vehicle` folder. The **Vehicle.class** file is within the `vehicle` subfolder and not in the same folder as the `.java` source file where it ordinarily would be placed.



If you cannot find the `com` folder on your storage device, you probably are not looking in the same folder where you compiled the class. Repeat Steps 4 and 5, but be certain that you first change to the command prompt for the directory where your source code file resides.

5. You could now delete the copy of the **Vehicle.java** file from the `VehiclePackage` folder (although you most likely want to retain a copy elsewhere). There is no further need for this source file in the folder you will distribute to users because the compiled `.class` file is stored in the `com\vehicleswesell\vehicle` folder. Don't delete the copy of your code from its original storage location; you might want to retain a copy of the code for modification later.

6. Open the **Sailboat.java** file in your text editor. For the first line in the file, insert the following statement:

```
package com.vehicleswesell.vehicle;
```

7. Save the file in the same directory as you saved **Vehicle.java**. At the command line, compile the file using the following command:

```
javac -d . Sailboat.java
```

Then you can delete the **Sailboat.java** source file from the `VehiclePackage` folder (not from its original location—you want to retain a copy of your original code).

8. Repeat Steps 6 and 7 to perform the same operations using the **Bicycle.java** file.
9. Open the **VehicleDatabase.java** file in your text editor. Insert the following statements at the top of the file:

```
import com.vehicleswesell.vehicle.Vehicle;
import com.vehicleswesell.vehicle.Sailboat;
import com.vehicleswesell.vehicle.Bicycle;
```

10. Save the file as **VehiclePackage\VehicleDatabase.java**. Compile the file, and then run the program. The program's output should be the same as it was before you added the import statements. Placing the `Vehicle`-related class files in a package is not required for the `VehicleDatabase` program to execute correctly; you ran it in exactly the same manner before you learned about creating packages.

Placing classes in packages gives you the ability to more easily isolate and distribute files.